

6 - ARQUIVOS

October 5, 2026

1 1 - Comandos Básicos

```
[ ]: # open(file, mode='r')
```

- 'r': Modo de leitura (padrão). Abre o arquivo para leitura. Gera erro se o arquivo não existir.
- 'w': Modo de escrita. Cria um novo arquivo ou sobrescreve o existente.
- 'x': Modo de criação. Cria um novo arquivo, mas gera erro se o arquivo já existir.
- 'a': Modo de anexação. Abre o arquivo para adicionar conteúdo ao final, sem sobrescrever os dados existentes.

Caso deseje combinar leitura com escrita, pode-se usar o simbolo '+':

- 'r+': Leitura e escrita. O arquivo deve existir.
- 'w+': Escrita e leitura. Cria ou sobrescreve o arquivo.
- 'a+': Anexação e leitura. Adiciona conteúdo ao final, mas permite leitura.

1.1 1.1 - Abrindo e Fechando um Arquivo

Isso irá gerar um erro pois o arquivo não existe.

```
[ ]: arquivo = open("alunos.txt", "r")

arquivo.close()
```

```
-----
FileNotFoundError                                Traceback (most recent call last)
Input In [40], in <cell line: 1>()
----> 1 arquivo = open("alunos.txt", "r")
      3 arquivo.close()

FileNotFoundError: [Errno 2] No such file or directory: 'alunos.txt'
```

Para evitar que esse erro ocorra, podemos verificar antes se o arquivo existe. O código abaixo usa uma função disponibilizada pela biblioteca "os" que vem de sistema operacional. Ao executar esse código pela primeira vez, o código contido no "else" será executado, afinal o arquivo ainda não existe e portanto será criado. Ao executar pela segunda vez, o código contido no "if" será executado, pois o arquivo já foi criado.

```
[ ]: import os

if os.path.exists("alunos.txt"):
    print("Arquivo existe!")
    arquivo = open("alunos.txt", "r")
    arquivo.close()
else:
    print("Arquivo não existe ainda!")
    arquivo = open("alunos.txt", "x")
    arquivo.close()
```

Arquivo não existe ainda!

1.2 1.2 - Lendo um Arquivo

Até agora você viu como abre e como fecha um arquivo. Vamos assumir que nenhum arquivo foi criado ainda. Você irá criar um arquivo manualmente chamado 'alunos.txt' e adicionar os seguintes aluno por linha.

Ana
Maria
José
Pedro

Se você quiser, ao invés de fazer isso manualmente, abra o terminal e digite o comando abaixo que ele fará essa tarefa de criar um arquivo e adicionar tais nomes.

```
[ ]: !echo -e "Ana\nMaria\nJosé\nPedro" > alunos.txt
```

Agora que já temos o arquivo "alunos.txt" criado, iremos ler esse arquivo. A função `read( )` é responsável por ler todo o conteúdo do arquivo de uma só vez.

```
[ ]: arquivo = open("alunos.txt", "r")

conteudo = arquivo.read()

arquivo.close()

print(conteudo)
```

Ana
Maria
José
Pedro

É possível também ao invés de ler todo o conteúdo, ler uma linha por vez. Isso faz sentido principalmente quando o conteúdo do arquivo é muito grande, fazendo com que a função que lê todo o arquivo - `read( )` - fique lenta.

```
[ ]: arquivo = open("alunos.txt", "r") #apenas cria o arquivo

linha1 = arquivo.readline()
linha2 = arquivo.readline()
linha3 = arquivo.readline()
print(linha1)
print(linha2)
print(linha3)

arquivo.close()
```

Ana

Maria

José

A função `readlines` com “s” no final ler também todo o arquivo e faz a separação por linhas, isto é, cada linha passa a ser um elemento da lista criada por essa função. Exemplo: `conteudo = [linha1, linha2, linha3, ..., linha n]`

```
[ ]: arquivo = open("alunos.txt", "r") #apenas cria o arquivo

conteudo = arquivo.readlines()

#for i in range(len(conteudo)):
#    conteudo[i] = conteudo[i].strip()

conteudo = [linha.strip() for linha in conteudo]

arquivo.close()

print(conteudo)
```

```
['Ana', 'Maria', 'José', 'Pedro']
```

1.3 1.3 - Escrevendo em um Arquivo

Ao abrir o arquivo no modo “a” de `append`, estamos adicionando algo ao final do arquivo. Isso é ideal quando você quer manter um histórico ou registrar informações continuamente, sem alterar os dados antigos.

```
[ ]: arquivo = open("alunos.txt", "a")

arquivo.write("\nNovo aluno 1")

arquivo.close()
```

```
[ ]: arquivo = open("alunos.txt", "r")

conteudo = arquivo.read()

arquivo.close()

print(conteudo)
```

Ana
Maria
José
Pedro
Novo aluno 1

Ao abrir o arquivo no modo “w”, todo conteúdo anterior é apagado e um novo conteúdo é escrito. Isso é ideal quando você quer garantir que o arquivo contenha apenas os dados mais recentes, descartando qualquer dado antigo.

```
[ ]: arquivo = open("alunos.txt", "w")

arquivo.write("\nNovo aluno 2")

arquivo.close()
```

```
[ ]: arquivo = open("alunos.txt", "r")

conteudo = arquivo.read()

arquivo.close()

print(conteudo)
```

Novo aluno 2

1.3.1 1.3.1 - Escrevendo e Lendo usando o comando ‘with’

O comando with em Python é usado para abrir arquivos de forma eficiente e segura. Ele garante que o arquivo seja automaticamente fechado após o bloco de código ser executado. Isso elimina a necessidade de fechar o arquivo manualmente com close().

```
[ ]: frase = "IFBA Campus Porto Seguro"
with open("meu_arquivo.txt", "w") as arquivo:
    arquivo.write(frase)

with open("meu_arquivo.txt", "r") as arquivo:
    conteudo = arquivo.read()
    print(conteudo)
```

1.3.2 1.3.2 - Funções Aprendidas

- `open()`
- `close()`
- `read()`
- `readline()`
- `readlines()`
- `write()`

2 2 - Salvando Listas ou Dicionários no Arquivo

A função `write()` só aceita strings como entrada. Um dicionário (dict) é uma estrutura de dados Python, não uma string, e, portanto, o método não sabe como manipulá-lo diretamente. Portanto, existem duas abordagens que podemos fazer:

Convertendo toda a estrutura em string

```
[22]: # Criando uma lista de itens
lista = ["Item 1", "Item 2", "Item 3", "Item 4"]

with open("meu_arquivo.txt", "w") as arquivo:
    arquivo.write(str(lista))

with open("meu_arquivo.txt", "r") as arquivo:
    conteudo = arquivo.read()
    lista = eval(conteudo)
    print(lista)
```

```
['Item 1', 'Item 2', 'Item 3', 'Item 4']
```

Iterando e salvando item a item

```
[23]: lista = ["Item 1", "Item 2", "Item 3", "Item 4"]

with open("meu_arquivo.txt", "w") as arquivo:
    for item in lista:
        arquivo.write(item + "\n")

with open("meu_arquivo.txt", "r") as arquivo:
    linhas = arquivo.readlines()
    lista_lida = [linha.strip() for linha in linhas]
    print(lista_lida)
```

```
['Item 1', 'Item 2', 'Item 3', 'Item 4']
```

3 3 - Usando o formato JSON

O JSON é uma excelente opção para salvar estruturas de dados como listas e dicionários, pois é simples, estruturado e ideal para garantir a interoperabilidade com outros sistemas ou linguagens. Apenas duas funções são necessárias:

- `dump()` para salvar os dados no arquivo
- `load()` para carregar os dados do arquivo

```
[24]: import json

# Dicionário ou lista de exemplo
dados = {"nome": "João", "idade": 25, "hobbies": ["futebol", "leitura"]}

# Salvar em um arquivo JSON
with open("meu_arquivo.json", "w") as arquivo:
    json.dump(dados, arquivo)

# Ler o arquivo JSON
with open("meu_arquivo.json", "r") as arquivo:
    dados_carregados = json.load(arquivo)

print(dados_carregados) # {'nome': 'João', 'idade': 25, 'hobbies': ['futebol', 'leitura']}
```

```
{'nome': 'João', 'idade': 25, 'hobbies': ['futebol', 'leitura']}
```

3.1 3.1 - Exemplo Real

```
[ ]: import json
import os

# Nome do arquivo JSON
FILENAME = "alunos.json"

# Função para carregar dados do arquivo JSON
def carregar_dados():
    if os.path.exists(FILENAME):
        with open(FILENAME, "r") as file:
            return json.load(file)
    else:
        #Iniciando com dados vazios
        return {}

# Função para salvar dados no arquivo JSON
def salvar_dados(dados):
    with open(FILENAME, "w") as file:
        json.dump(dados, file, indent=4)
```

```

# Função para adicionar ou editar um aluno
def adicionar_ou_editar_aluno(dados):
    numero = input("Digite o número de matrícula: ")
    nome = input("Digite o nome do aluno: ")
    idade = input("Digite a idade do aluno: ")
    curso = input("Digite o curso do aluno: ")

    # Atualiza ou adiciona o aluno no dicionário
    dados[numero] = {
        "nome": nome,
        "idade": idade,
        "curso": curso
    }
    print(f"Aluno {numero} adicionado/atualizado com sucesso!")

# Função para exibir informações de um aluno
def exibir_aluno(dados):
    numero = input("Digite o número de matrícula do aluno que deseja consultar: ")
    if numero in dados:
        print("Informações do aluno:")
        for chave, valor in dados[numero].items():
            print(f"{chave.capitalize()}: {valor}")
    else:
        print("Aluno não encontrado.")

# Função para remover um aluno
def remover_aluno(dados):
    numero = input("Digite o número de matrícula do aluno que deseja remover: ")
    if numero in dados:
        del dados[numero]
        print(f"Aluno {numero} removido com sucesso!")
    else:
        print("Aluno não encontrado.")

# Função principal do programa
def main():
    dados = carregar_dados()

    while True:
        print("\nMenu:")
        print("1. Adicionar ou editar aluno")
        print("2. Exibir informações de um aluno")
        print("3. Remover aluno")
        print("4. Salvar e sair")

        opcao = input("Escolha uma opção: ")

```

```

    if opcao == "1":
        adicionar_ou_editar_aluno(dados)
    elif opcao == "2":
        exibir_aluno(dados)
    elif opcao == "3":
        remover_aluno(dados)
    elif opcao == "4":
        salvar_dados(dados)
        print("Dados salvos. Saindo do programa.")
        break
    else:
        print("Opção inválida. Tente novamente.")

if __name__ == "__main__":
    main()

```

4 4 - Usando o formato Pickle

O `pickle` é um módulo da biblioteca padrão do Python usado para salvar objetos Python em arquivos. Diferente do JSON, ele não salva os dados em texto legível: ele salva em formato binário.

Isso é útil quando queremos guardar objetos Python mantendo seus tipos originais, como `date`, listas, dicionários, tuplas e outros objetos. A desvantagem é que o arquivo gerado não foi feito para ser lido por pessoas em um editor de texto.

4.1 4.1 - JSON: arquivo de texto, mas tipos especiais precisam de conversão

O JSON é ótimo para trocar dados com outros sistemas porque o arquivo fica em texto legível. Porém, ele não conhece todos os tipos do Python. Por exemplo, uma data precisa ser convertida para texto antes de ser salva.

Ao ler o arquivo de volta, a data continua sendo texto. Se quisermos recuperar o tipo `date`, precisamos aplicar um conversor.

```

[ ]: import json
    from datetime import date

    aluno = {
        "nome": "Maria",
        "matricula": "2024001",
        "data_nascimento": date(2007, 5, 14)
    }

    # O JSON não salva date diretamente.
    # Por isso, convertemos a data para string antes de salvar.
    aluno_json = {
        "nome": aluno["nome"],

```

```

    "matricula": aluno["matricula"],
    "data_nascimento": aluno["data_nascimento"].isoformat()
}

with open("aluno.json", "w") as arquivo:
    json.dump(aluno_json, arquivo, indent=4)

with open("aluno.json", "r") as arquivo:
    aluno_lido = json.load(arquivo)

print(aluno_lido)
print(type(aluno_lido["data_nascimento"]))

```

Observe que `data_nascimento` foi lida como `str`, ou seja, como texto. Para voltar a ser uma data, precisamos converter manualmente.

```

[ ]: from datetime import date

aluno_lido["data_nascimento"] = date.
    ↳fromisoformat(aluno_lido["data_nascimento"])

print(aluno_lido)
print(type(aluno_lido["data_nascimento"]))

```

4.2 4.2 - Pickle: arquivo binário que preserva os tipos do Python

Com `pickle`, o objeto é salvo em binário. Como ele foi feito para guardar objetos Python, o tipo `date` não é perdido. Ao ler o arquivo, não precisamos converter a data de volta manualmente.

Para trabalhar com arquivos binários, usamos a letra `b` no modo de abertura do arquivo:

- `wb`: write binary, ou seja, escrever em binário;
- `rb`: read binary, ou seja, ler em binário.

O `b` vem de **binary**.

```

[ ]: import pickle
    from datetime import date

aluno = {
    "nome": "Maria",
    "matricula": "2024001",
    "data_nascimento": date(2007, 5, 14)
}

with open("aluno.pkl", "wb") as arquivo:
    pickle.dump(aluno, arquivo)

with open("aluno.pkl", "rb") as arquivo:

```

```
aluno_lido_pickle = pickle.load(arquivo)

print(aluno_lido_pickle)
print(type(aluno_lido_pickle["data_nascimento"]))
```

4.3 4.3 - Arquivo binário não é feito para leitura direta

Como o Pickle salva os dados em binário, o arquivo `.pkl` não fica compreensível como um JSON. Podemos abrir alguns bytes do arquivo apenas para perceber que seu conteúdo não é texto comum.

```
[ ]: with open("aluno.pkl", "rb") as arquivo:
      conteudo_binario = arquivo.read(40)

print(conteudo_binario)
```

4.4 4.4 - Resumo: JSON x Pickle

- JSON salva dados em texto e é fácil de abrir e ler.
- JSON é melhor quando os dados precisam ser usados por outros sistemas ou linguagens.
- JSON pode exigir conversores para tipos como `date`.
- Pickle salva dados em binário, usando `wb` para escrever e `rb` para ler.
- Pickle preserva tipos do Python, então uma `date` continua sendo `date` depois da leitura.
- Pickle é mais indicado para salvar e carregar objetos Python dentro de projetos Python.